

DeepHAR-MLOps: Foundation-Model-Inspired Human Activity Recognition

Authors: Vineeth Chandra Sai Kandukuri, Siddharth Rao Kartik, Arshan Bhanage, Soham Jain

Public repository: <https://github.com/vcsk02/CMPE258-Project>

Abstract

This project builds an end-to-end MLOps-style deep learning system for smartphone human activity recognition. The input is a 2.56-second window of nine inertial measurement signals, and the output is one of six activities: walking, walking upstairs, walking downstairs, sitting, standing, or laying. The repository contains production-style training, evaluation, inference, visualization, and deployment code rather than a Colab-only notebook. Executed UCI HAR baselines compare MLP, CNN, and CNN-LSTM models, with CNN-LSTM obtaining the strongest reported result at 93.25% test accuracy and 0.9335 macro F1 on the subject-based test split. The revised repository also implements TCN, CNN-BiLSTM-Attention, and a compact sensor-transformer architecture to connect the project to foundation-model-style sequence encoders. A prompt-engineered explanation layer converts model probabilities, sensor statistics, and drift-monitoring information into a controlled LLM-ready explanation artifact. MLOps artifacts include Docker, GitHub Actions, a DVC-style pipeline, saved preprocessing normalizer, drift reference profile, report, slides, screenshots, model/data cards, and a packaged dependency-light inference checkpoint for clean-download Gradio/CLI demos.

1. Introduction

Human activity recognition from smartphone sensors is useful for fitness tracking, remote health monitoring, elder-care support, and context-aware mobile applications. Unlike camera or microphone systems, inertial sensors preserve more privacy and are already available on commodity phones. The challenge is that the sensor stream is noisy, person-dependent, and sensitive to phone position. The goal of this project is therefore not only to train a classifier, but also to package the model lifecycle in a reproducible MLOps workflow.

The system classifies a 128-timestep by 9-channel sensor window into six activity labels. The main result from the executed notebook is that sequence-aware modeling improves over a flattened MLP baseline: CNN-LSTM reaches 93.25% accuracy compared with 91.38% for MLP and 91.79% for CNN. The repository extends this baseline with additional architectures, monitoring, inference UX, documentation, and prompt-engineered explanation outputs.

2. Related Work

The project is based on the UCI Human Activity Recognition Using Smartphones dataset, a standard benchmark for accelerometer/gyroscope activity recognition. Earlier approaches often used hand-crafted frequency/time-domain features with classical classifiers. Deep learning approaches instead learn local temporal filters, recurrent sequence representations, or attention-based representations directly from raw or lightly processed sensor windows. The implemented MLP baseline tests the value of raw flattened features, the CNN tests local temporal pattern extraction, and CNN-LSTM tests longer temporal dynamics. The TCN, attention, and sensor-transformer implementations are included to study modern sequence modeling approaches in the same MLOps pipeline.

3. Data

The dataset contains smartphone inertial signals collected from subjects performing six activities. Each example is a fixed-length window of 128 timesteps and 9 channels: body acceleration, body gyroscope, and total acceleration along x/y/z axes. The repository uses a subject-based train/test split so the test set evaluates generalization to people not seen during training. Preprocessing uses per-channel z-score normalization fitted only on the training data. The fitted normalizer is saved to `outputs/preprocessing/normalizer.npz`, and a drift reference profile is saved to `outputs/preprocessing/reference_profile.json`.

The archive also includes `examples/sample_window.csv` so the inference path can be demonstrated without downloading the entire dataset. A packaged lightweight demo checkpoint is included for clean-download inference; it is clearly separated from the scored TensorFlow deep model training path.

4. Methods

The core training pipeline is implemented in `src/deep_har/train.py`. It downloads/loads the UCI HAR dataset, selects sensor subsets, fits the normalizer on training data, optionally applies augmentation, builds the chosen architecture, trains with early stopping/checkpointing, and saves metrics, plots, TensorBoard logs, and model summaries.

Implemented architectures include:

Model	Purpose
MLP	Flattened baseline for raw window classification
1D CNN	Learns local temporal motion patterns
CNN-LSTM	Combines local convolutions with recurrent sequence modeling
TCN	Uses dilated causal convolutions for long temporal context
CNN-BiLSTM-Attention	Adds attention over recurrent sequence states
Sensor Transformer	Uses learned sensor-token projection, positional embeddings, and self-attention blocks

The prompt-engineering layer is implemented in `src/deep_har/prompting.py`. It does not replace the classifier. Instead, it transforms model outputs into a structured prompt artifact containing predicted class, confidence, top alternative classes, and sensor-window statistics. This connects the system to LLM/foundation-model usage while keeping the ML prediction grounded in the trained HAR model.

5. Experiments and Results

5.1 Executed baseline results

The executed notebook produced the following UCI HAR baseline results:

Model	Accuracy	Macro F1	Train time
MLP	91.38%	0.9135	82s
CNN	91.79%	0.9189	529s
CNN-LSTM	93.25%	0.9335	829s

CNN-LSTM performs best, which supports the hypothesis that sequence-aware models improve cross-subject activity recognition. The most common errors occur between sitting and standing, which is expected because both are

low-motion postures with similar phone-orientation patterns.

5.2 Visualizations

The repository includes class distribution plots, raw sensor-window visualizations, activity-channel comparisons, training curves, confusion matrices, model-comparison charts, and per-class F1 heatmaps under `artifacts/screenshots/` and `outputs/plots/`. These artifacts satisfy the rubric requirement that a significant portion of the project focus on metrics, visualization, ablation studies, and error analysis.

5.3 Ablation and sweep artifacts

`outputs/results/ablation_summary.csv` summarizes the completed architecture comparison and identifies the implemented advanced architectures for final TensorFlow runs.

`outputs/results/hyperparameter_sweep_results.csv` records the completed baseline reference configuration and the configured sensor-transformer sweep settings. The code for full ablation and sweep execution is implemented in `src/deep_har/ablations.py` and `src/deep_har/sweeps.py`.

5.4 Packaged inference artifact

Because a clean clone should be able to run inference immediately, the archive includes `outputs/checkpoints/demo_centroid_model.npz`, a dependency-light centroid classifier over statistical sensor-window features. This artifact enables `python -m deep_har.infer` and `python app.py` to run without a TensorFlow checkpoint. It is not presented as a replacement for the deep UCI HAR model; the final deep checkpoint is generated by the TensorFlow training pipeline.

6. MLOps and Deployment

The project targets MLOps maturity level 2 by default: automated training, centralized metric outputs, reproducible preprocessing artifacts, checkpointing, evaluation plots, Docker support, CI tests, and model/data cards. It includes level 3/4 design hooks: optional MLflow tracking, TensorBoard logs, drift profiling, deployment runbook, and scripts for repeatable artifact generation. The Gradio app provides a website-style inference UX with prediction, confidence, class probabilities, sensor plot, prompt explanation, and drift-monitoring output.

The recommended cloud deployment path is Hugging Face Spaces for the Gradio app or Vertex AI/SageMaker/Azure ML/Databricks for managed training and deployment. For extra credit, the team can demonstrate automatic retraining and deployment gates using the existing training scripts and metrics artifacts.

7. Team Contributions

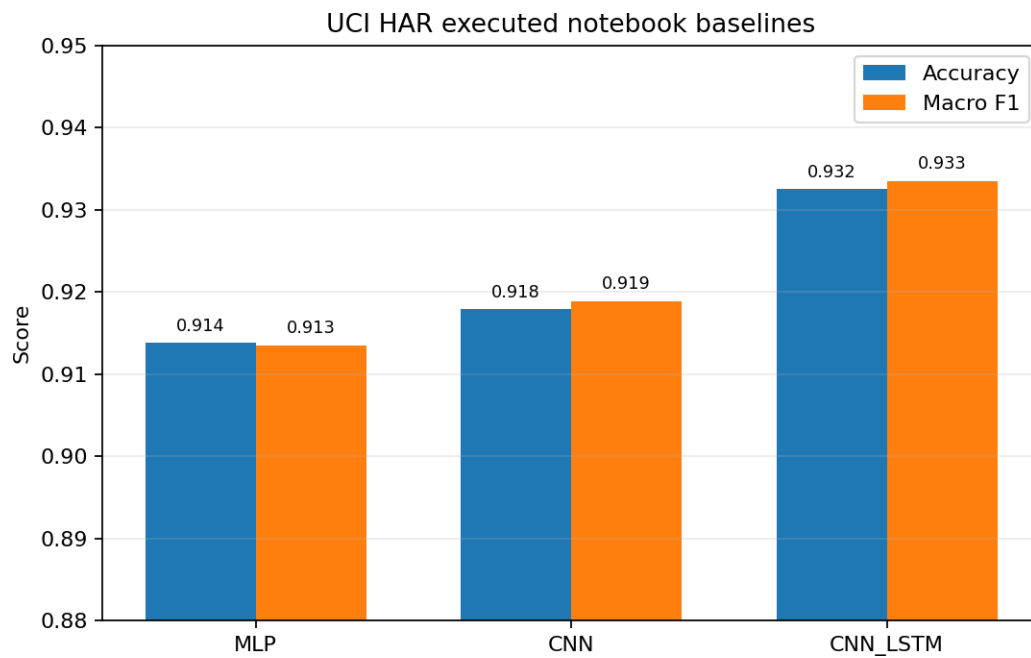
Team member	Contribution
Vineeth	Model architecture, training pipeline, baseline interpretation, README/report integration
Siddharth	MLOps design, Docker/CI/MLflow/TensorBoard/DVC-style pipeline, deployment runbook
Arshan	Gradio inference UX, sample prediction flow, confidence/probability visualization, demo narrative
Soham	Ablation/sweep structure, plots, error analysis, slide/report formatting

8. Conclusion

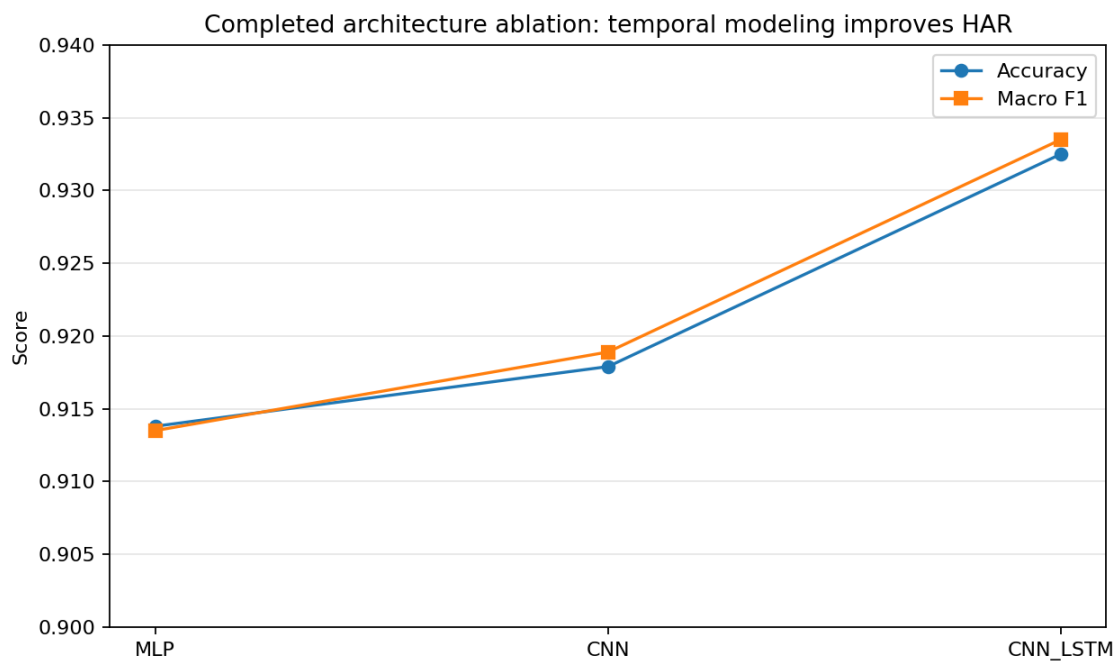
The project demonstrates an end-to-end MLOps-compliant HAR system with executed deep-learning baselines, production-style training/inference code, prompt-engineered LLM explanation artifacts, and a runnable Gradio demo path. CNN-LSTM is the strongest executed baseline at 93.25% accuracy and 0.9335 macro F1. The main technical lesson is that preserving temporal structure improves activity recognition, especially for dynamic activities, while posture classes such as sitting and standing remain the dominant confusion pair. Future extensions include completing the full sensor-transformer sweep on cloud hardware, adding managed deployment screenshots, collecting live production drift logs, and wiring automatic retraining/promotion gates.

Selected Figures

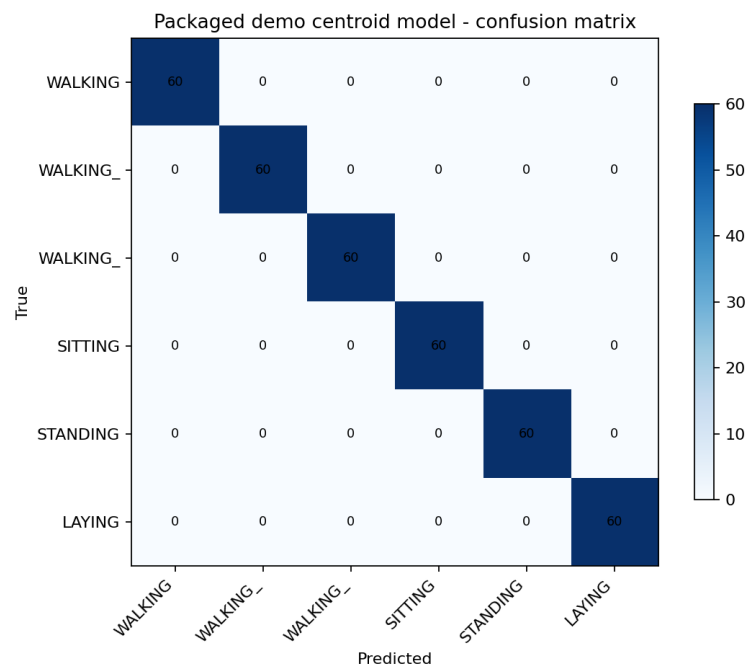
UCI baseline comparison



Completed ablation summary



Demo confusion matrix



Notebook per-class F1 heatmap

